

MANAGING AND INTERACTING WITH DOCKER CONTAINERS

1. What is a Docker Container?

- A Docker container is a lightweight, portable package that contains everything needed to run an application - including the code, runtime, system tools, libraries, and settings.
- Containers run on the Docker Engine and are isolated from the host and other containers.

2. Common Docker Container Commands

- These commands help create, manage, interact with, and troubleshoot containers.
 - ✓ **docker run**
- **Purpose:** Creates and starts a new container from an image
 - ✓ **Syntax:**
 - `docker run [OPTIONS] IMAGE [COMMAND]`

3. Common Options:-

- **-d** → Run in detached (background) mode.
- **-p** → Map host port to container port (e.g. `-p 8080:80`).
- **-P** → Randomly map all exposed container ports to host ports.
- **-e** → Set environment variables (e.g. `-e VAR=value`).
- **--name** → Assign a name to the container.

Example:- `docker run -d -p 8080:80 --name my-container`

4. docker create

- **Purpose:** Creates a container from an image without starting it.
- **Syntax:** `docker create --name image_name`

5. docker start / docker stop

- `docker start <container>` → Starts a stopped container.
- `docker stop <container>` → Stops a running container gracefully.

6. docker rm

- **Purpose:** Removes a stopped container.
- **Example:** `docker rm <container>`
- Use `docker stop` before removing a running container

7. docker ps

- **Purpose:** Lists running containers.
- ✓ **Options:-**
 - **-a:** Show all containers (running and stopped).
 - **-q:** Show only container IDs.
- **Examples:**
 - `docker ps`
 - `docker ps -a`
 - `docker ps -q`

8. docker exec

- **Purpose:** Runs a command inside a running container.
- **Syntax:** `docker exec -it <container_name> <command>`
- **Example:** `docker exec -it web bash`
- This opens an interactive terminal inside the container

9. docker inspect

- **Purpose:** Displays detailed information (in JSON format) about a container's configuration and state
- **Example:** `docker inspect container`

10. docker cp

- **Purpose:** Copies files or directories between the container and the host.
- `docker cp ./index.html web:/usr/share/nginx/html/`

11. docker logs container

- **Purpose:** Displays logs from a container's

12. docker stats

- ❖ **Purpose:** Shows real-time metrics of container usage (CPU, memory, network I/O).

▪ Exposing Ports with -p Option

- The `-p` flag is used in `docker run` to publish container ports to the host.
- **Syntax:**

`docker run -p <host_port>:<container_port> image_name`

Example: `docker run -d -p 8080:80 nginx`

- 80 is the internal container port (Nginx default)

❖ Inspects and Troubleshooting Steps in Docker

- Docker provides several commands to inspect container state and troubleshoot issues.
- **a) docker inspect**
- Displays detailed configuration and runtime information
- `docker inspect`
- Useful to check:
 - IP address Mounted
 - volumes Network mode & Port mapping

❖ docker logs

`docker logs <container_name>`

❖ docker stats

Real-time metrics for containers (CPU, memory, network, I/O).

d) docker cp

- Copies files between host and container.
- `docker cp ./path/in/container /path/on/host`

❖ Common Docker Commands with Examples

Command	Purpose	Example
<code>docker run</code>	Create and start container <i>x</i>	<code>docker run -d -p 8080:80 nginx</code>
<code>-d</code>	Detached mode	Run in background
<code>-p</code>	Port mapping	<code>-p 8080:80</code>
<code>-P</code>	Publish all exposed ports to random host ports	<code>-P</code>
<code>-e</code>	Set environment	<code>variable-e ENV=dev</code>
<code>--name</code>	Assign container name	<code>--name webserver</code>
<code>docker create</code>	Create container without starting	<code>docker create --name myapp alpine</code>
<code>docker start</code>	Start existing container	<code>docker start myapp</code>
<code>docker stop</code>	Stop running container	<code>docker stop myapp</code>
<code>docker rm</code>	Remove stopped container	<code>docker rm myapp</code>
<code>docker ps</code>	List running containers	<code>docker ps</code>
<code>docker ps -a</code>	List all containers	<code>docker ps -a</code>
<code>docker ps -q</code>	Show only container IDs	<code>docker ps -q</code>
<code>docker exec -it</code>	Run commands in container	<code>docker exec -it web bash</code>
<code>docker inspect</code>	Detailed info in JSON	<code>docker inspect web</code>
<code>docker cp</code>	Copy files docker	<code>docker cp cp web:/app/file.txt ./</code>
<code>docker logs</code>	View container logs	<code>docker logs web</code>
<code>docker stats</code>	Monitor resource usage	<code>docker stats</code>

❖ Troubleshooting Workflow

Step	Tool/Command	Purpose
Check logs	<code>docker logs</code>	View application errors or output
Check status	<code>docker ps</code> , <code>docker inspect</code>	Ensure container is running and healthy
Enter container	<code>docker exec -it</code>	Inspect filesystem or environment
Copy config/logs	Copy config/logs	Transfer files for deeper analysis
Monitor performance	<code>docker stats</code>	Diagnose CPU or memory issues